Java Based Business Application

Francisco Acosta

CIS355A Final Project Presentation | December 2024

Introduction

Java is often referred to as the internet’s programming language due to its relevance across the web.

This project creates a Java GUI application for a fictitious landscaping organization to keep track of clienteles' personal information and costs associated with work to be done with the integration of a SQL database.

The goal is to provide an efficient and visually pleasing Java application with a smooth user experience.

Follow my journey through the stages of planning, designing and development.

Design Phase

Planning

The design phase consisted of creating a GUI wireframe diagram of the application within Microsoft Visio. The wireframe diagram created a visual representation of the application, making it easier to develop with a target to achieve and a reference point to utilize as a guide.

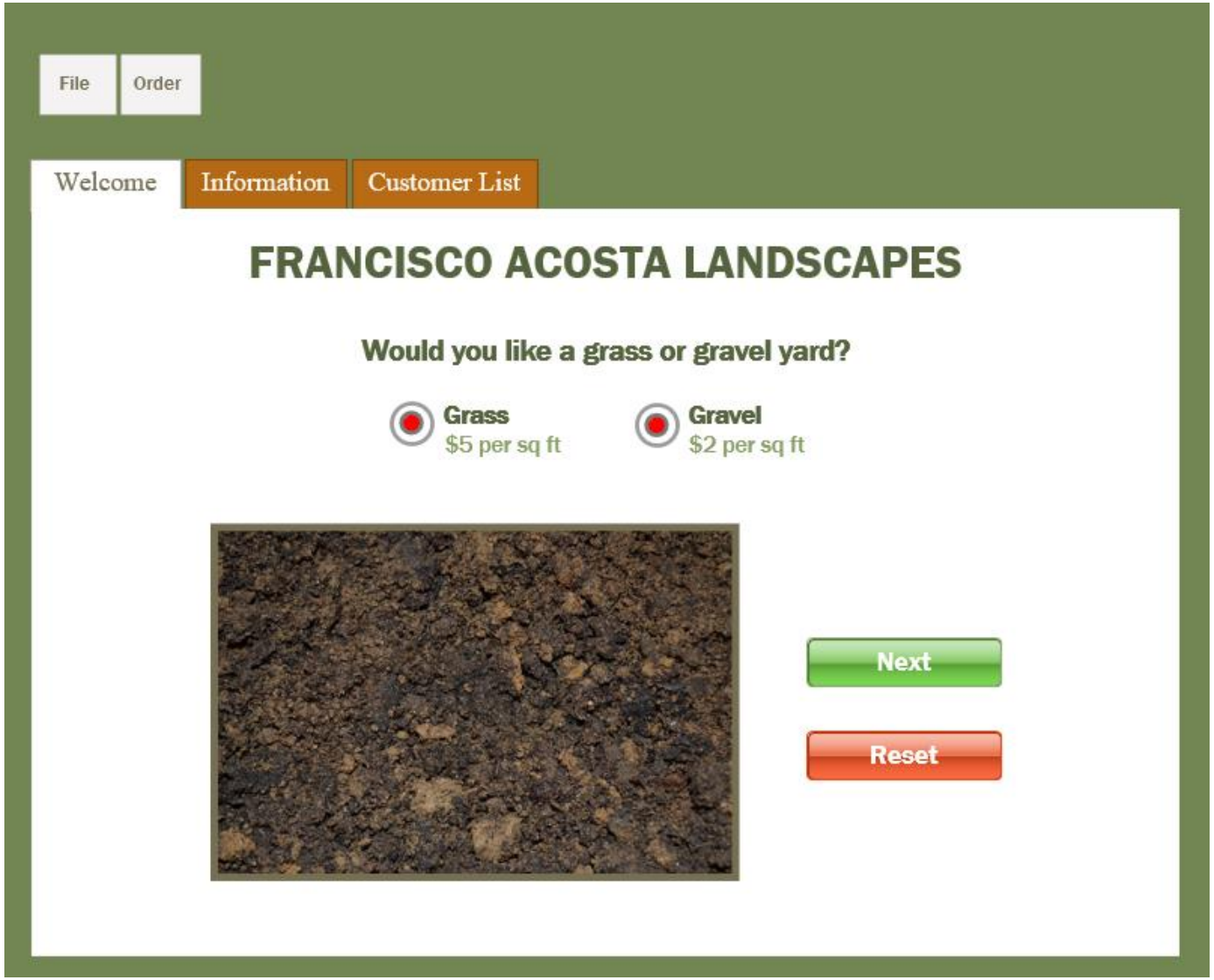
A UML class diagram was also created within Microsoft Visio.
This diagram created an efficient way to create this application by outlining classes, as well as the structure of the application.

Both diagrams were critical in saving time by giving the fictitious client the ability to view the application before the development process begun, therefore generating an overview of the application.

Wireframe Diagram

This diagram served as a blueprint of the application development process.

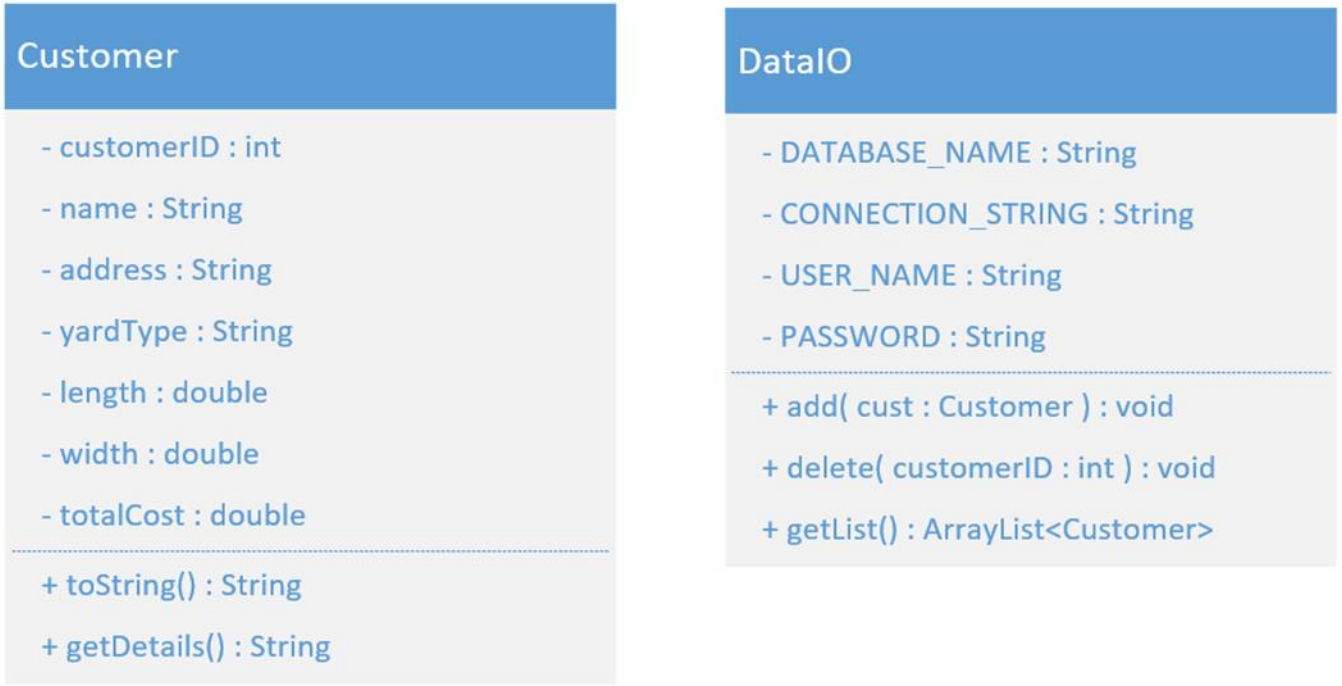
It includes the layout of the application and the associated components that will be created during the development of the application.



UML Class Diagram

The UML class diagram visualizes the overall structure of the application.

This includes all classes, attributes, methods and relations between objects.



Simple GUI Application

GUI Build

The first step to starting the development of this application was to create a simple GUI that could be built upon as the development process advances.

This initial build brought the UML class diagram to code with the creation of a client input form that incorporates labels, text fields, events, and buttons within a centered application frame.

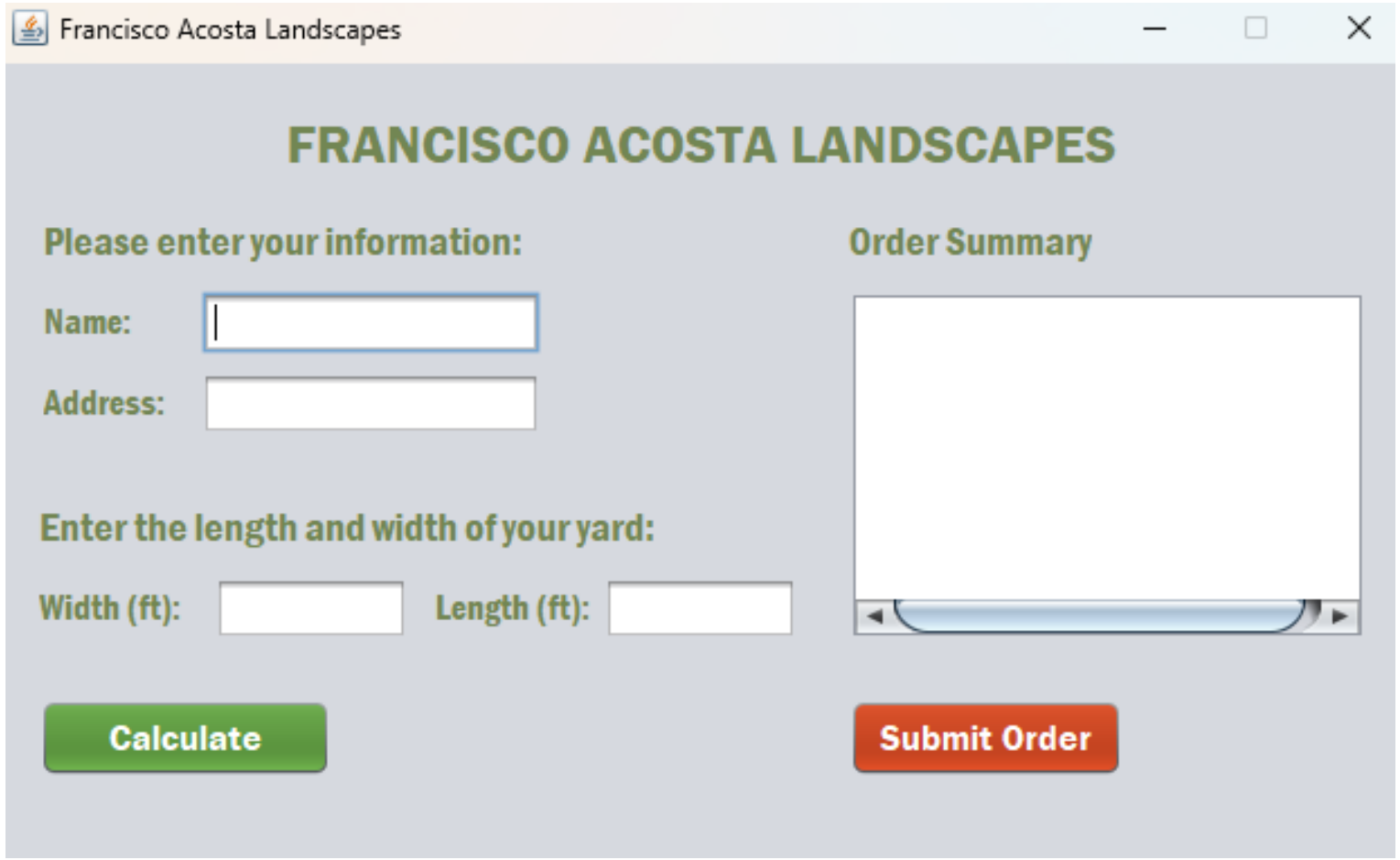
Fields included user inputs for a customer’s name, address, width and length of the client’s yard, which incorporated data hiding techniques for security purposes.

A new class known as “Customer” was created to keep track of field input and calculate the cost of service which is assigned to the “totalCost” attribute.
Calculations are based on a predefined cost value multiplied by the area of the client's yard size.

Additional Implementations

- An “Order Summary” text area was created to display the collection of user inputs, as well as the total cost of work.
- A “Calculate” button was created with an event that executes the calculations of the total cost of work and adds all user input, as well as the cost of work to the “Order Summary” text area.
- Validation was incorporated through exception handling.
  - Checks to see if the width and length fields are a numeric value.
  - Checks to see if the address is a string with a least six characters.
  - Checks if any input fields are blank.
- The “customerID” attribute, “yardType” attribute, and “Submit Order” button were implemented for future development.

Information GUI Form



Information Form Code

```
//Checks if name field is empty
if (sName.equals(""))
{
    JOptionPane.showMessageDialog(this, "Enter a Name.", "Name Error", JOptionPane.ERROR_MESSAGE);
    txtName.requestFocusInWindow();
    return false;
}

//Checks if address field is empty
if (sAddress.equals(""))
{
    JOptionPane.showMessageDialog(this, "Enter an address.", "Address Error", JOptionPane.ERROR_MESSAGE);
    txtAddress.requestFocusInWindow();
    return false;
}

//Checks if address is greater than 6 characters
if (sAddress.length() <= 5)
{
    JOptionPane.showMessageDialog(this, "Address isn't long enough.", "Address Error", JOptionPane.ERROR_MESSA
    txtAddress.requestFocusInWindow();
    return false;
}

//Checks if width field is empty
if (sWidth.equals(""))
{
    JOptionPane.showMessageDialog(this, "Enter a width.", "Width Error", JOptionPane.ERROR_MESSAGE);
    txtWidth.requestFocusInWindow();
    return false;
}

//Exception: Checks if width field is a numberic value and clears the field if it isn't
try
{
    Double.parseDouble(sWidth);
}
catch (NumberFormatException e)
{
    JOptionPane.showMessageDialog(this, "Width must be a number.", "Width Error", JOptionPane.ERROR_MESSAGE);
```

Advanced GUI Controls

Creation of Welcome Form GUI

This phase of the build was dedicated to creating an additional GUI form that would serve as the welcome screen of the application and give the user an opportunity to select what type of landscape work is needed.

The options of grass and gravel were made available by the creation of radio buttons. The selection of one of these buttons would therefore alter the total cost based on the type of work to be done.

Pricing for grass is $5 per square feet and the price for gravel is $2 per square feet. Associated imagery was also incorporated for a visual representation of what type of work will be done and the selection of a radio button would dictate the imagery displayed.

Additional GUI Components

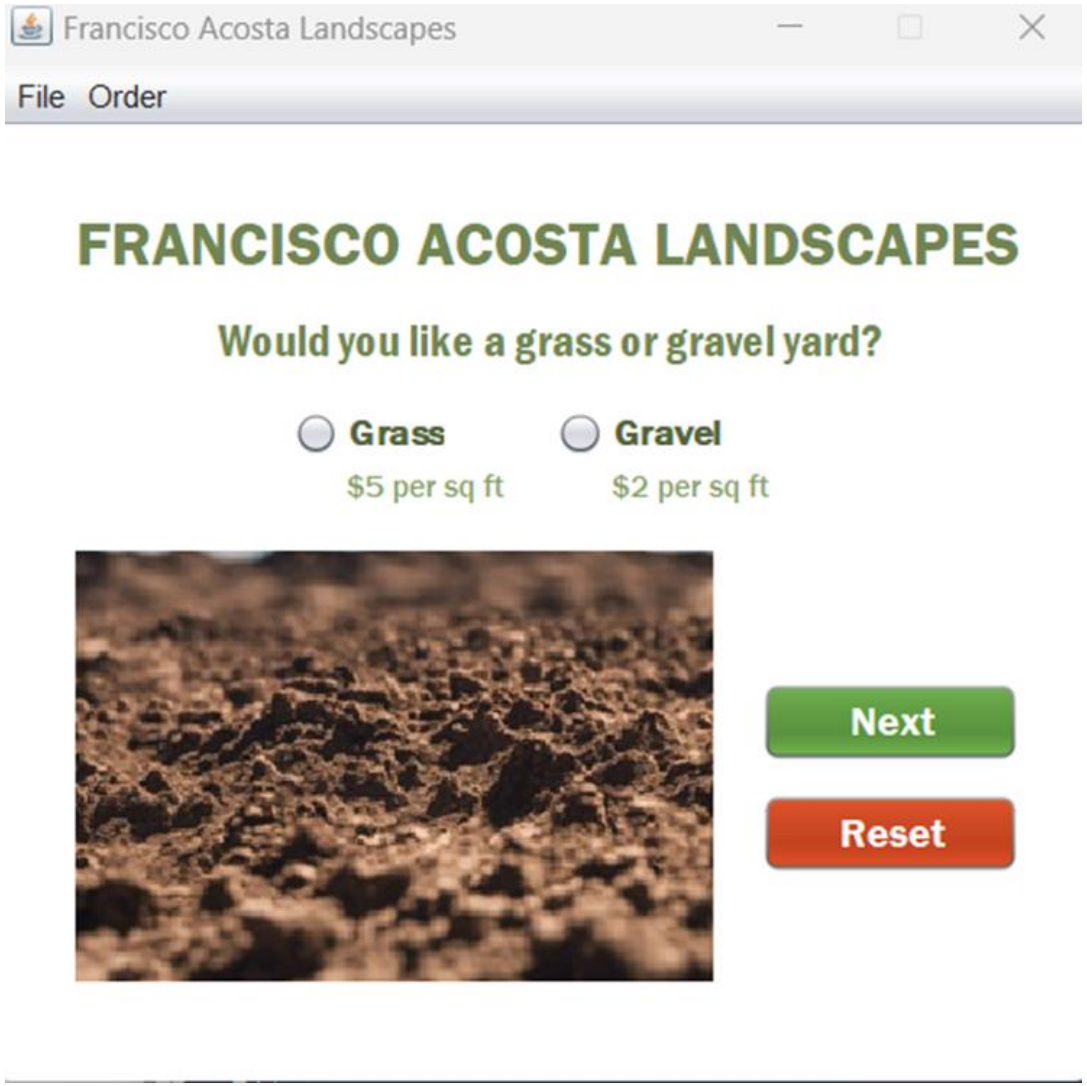
A “Reset” button was added to return the radio buttons to their default state of no selection and update the accompanying imagery.

A menu was also incorporated into this GUI form.
A “Reset” menu item was added with the same functionality as the “Reset” button and an “Exit” menu item was added to exit the application.
A “Submit Order” menu item was also incorporated for future development.

Finally, a “Next” button was created with an event to launch the “Information” form if a selection was made.

Exception handling was integrated into the application to display an error message and prevent the progression to the “Information” form if no selection was made.

Completed Welcome Form



Welcome Form Code

```
private void btnResetActionPerformed(java.awt.event.ActionEvent evt) {
    reset( );
}

private void rdoGrassActionPerformed(java.awt.event.ActionEvent evt) {
    if (rdoGrass.isSelected())
    {
        // Looks in directory where class exists, and gets location of "grass.jpg"
        URL location = this.getClass().getResource("grass.jpg");

        // Creates image using the location of "grass.jpg"
        ImageIcon icon = new ImageIcon(location);

        // Sets the image icon to the label
        lblPhoto.setIcon(icon);
    }
}

private void rdoGravelActionPerformed(java.awt.event.ActionEvent evt) {
    if (rdoGravel.isSelected())
    {
        // Finds image, creates image icon, and sets image icon to label
        lblPhoto.setIcon( new ImageIcon( this.getClass().getResource("gravel.jpg") ) );
    }
}

private void btnCalculateActionPerformed(java.awt.event.ActionEvent evt) {
    // validate the inputs
    if (validateInputs() == false)
    {
        // End the method if validation failed
        return;
    }

    // Creates the Customer object and shows information
    Customer cust = createCustomer();
    txaOrderSummary.setText( cust.getDetails() );
```

Final GUI Form

Customer List GUI Build

The third and final GUI form was developed for the management of the user's list of customers.

GUI components included:

- A company name Jlabel.
- A customer list JLabel and JList which displays the name of customers from the "Information" form.
- A customer details JLabel and textArea that displays all details from the "Information" form.
- A "Load List" Button for future development.
- A "Delete Customer" button to remove customers from the Jlist.
- Tabs to link and cycle through all three forms within this application.

Added Functionality

The “Submit” button and menu item were both updated to add data from the “Information” form fields and radio button selection from the “Welcome” form into the customer Jlist and details text area.

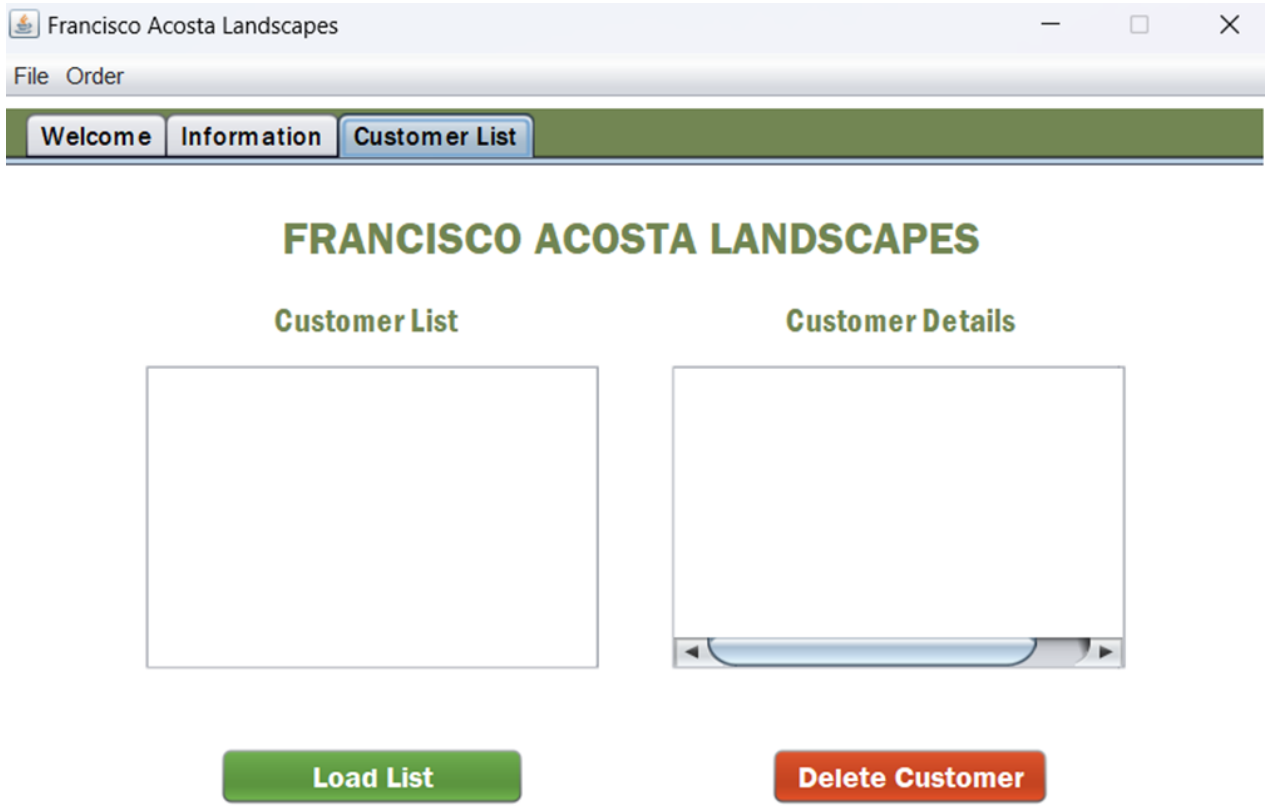
Validation through exception handling was incorporated to make sure both the “Welcome” and “Information” forms have the proper inputs.

- Selecting a radio button for the yard type will allow the next button to progress the user to the “Information” form.
- Validation of inputs from the “Information” form
  - Checks if fields are blank
  - Checks if fields have the correct value type.
    - The width and length fields must be a numeric value.
    - The address must be more than 6 characters.

Additional Functionality

- The “Reset” button on welcome page and the “Reset” menu item functionality was updated to reset both the welcome form radio buttons and information form fields.
- The “Customer Details” text area utilizes an ArrayList to process and mirror data from the “Information” form’s “Order Summary” text area.
- The “Information” form’s “Submit Order” button and the “Submit Order” menu item are both updated to add data to the “Customer List” form’s Jlist and text area.

Customer List Form



Customer List Code

```
// Builds customer order details
private Customer createCustomer()
{
    String name = txtName.getText();
    String address = txtAddress.getText();
    double width = Double.parseDouble(txtWidth.getText());
    double length = Double.parseDouble(txtLength.getText());
    String yardType = "";
    double totalCost = 0.0;

    if (rdoGrass.isSelected())
    {
        yardType = "Grass";
        totalCost = GRASS_PER_SQFT * width * length;
    }
    else if (rdoGravel.isSelected())
    {
        yardType = "Gravel";
        totalCost = GRAVEL_PER_SQFT * width * length;
    }
    else
    {
        JOptionPane.showMessageDialog(this,
                "Error. Please select a yard type.");
        totalCost = 0.0;
    }

    Customer cust = new Customer(0, name, address, yardType,
    length, width, totalCost);
    return cust;
}
// Adds order to customer list
private void submitOrder()
{
    if (validateInputs() == false)
    {
    return;    // end the method if validation failed
    }

    Customer cust = createCustomer();
```

Read and Write to File

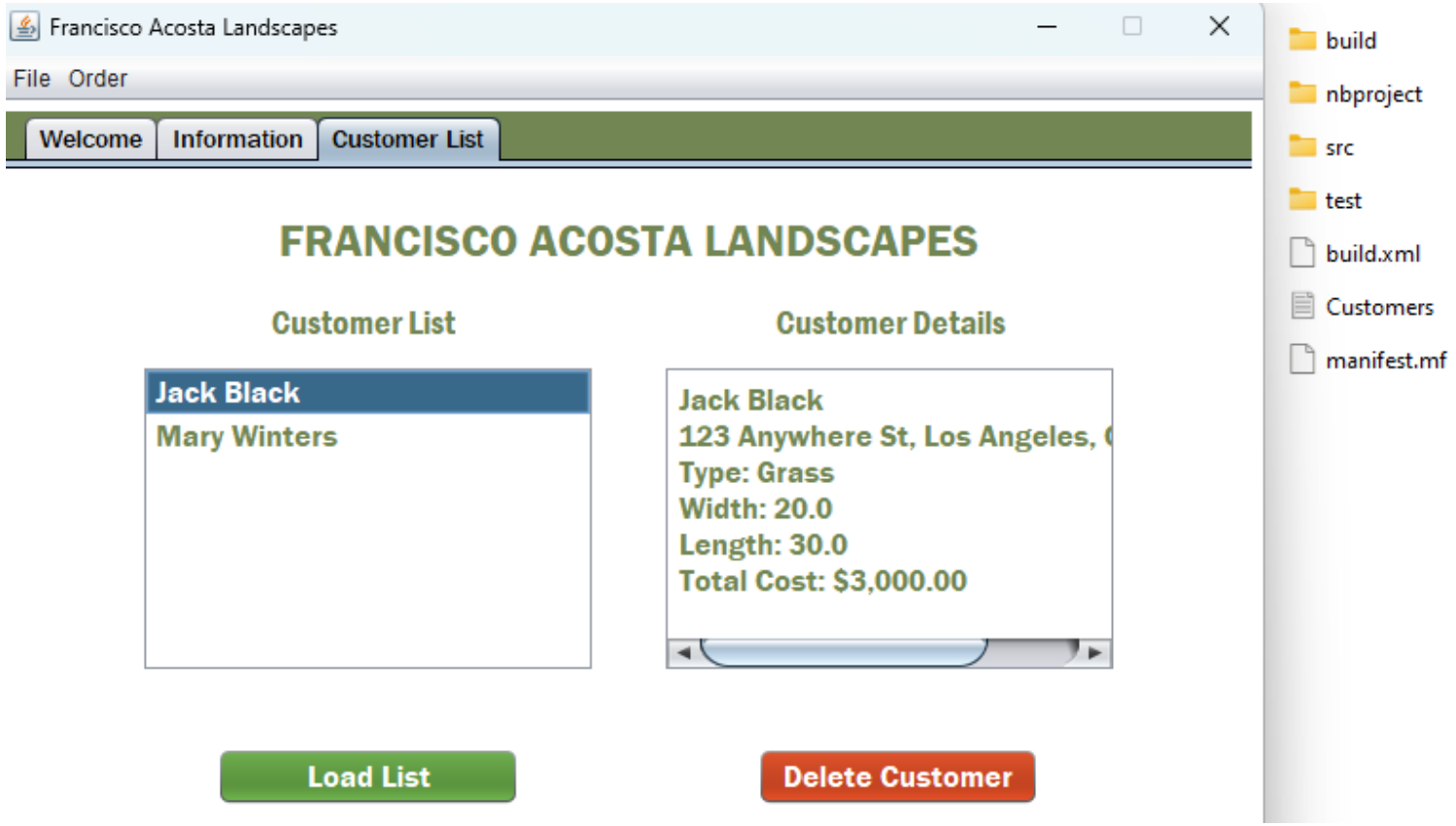
DataIO Class and Added Events

A new DataIO class was created to read from and write to a file with the GUI. This data would be displayed on the “Customer List” form’s Jlist and text area.

The following updated functionality was incorporated:

- An event was created for the “Information” form’s “Submit Order” button and “Submit Order” menu item to write data to a new txt file or overwrite an existing one.
  Data is populated by the “Information” form’s input fields and “Welcome” form’s radio button selection.
- An event was created for the “Customer List” form’s “Load List” button to read a txt file.
- The “Customer List” form’s “Delete Customer” button was updated to remove clients from a txt file.

Customer List with Read and Write Functionality



Local File Read and Write Code

Customer.java | DataIO.java | LandscapeGUI.java

Source | History

```
68
69      public ArrayList<Customer> getList() throws FileNotFoundException, IOException
70      {
71          // Gets Customer objects from the file and is returned as ArrayList
72          // Creates an arraylist
            ArrayList<Customer> customers = new ArrayList<Customer>();
74
75          // Reads data from the file
            BufferedReader inbuffer = new BufferedReader(new FileReader("Customers.txt"));
77          StringTokenizer tokens;
78
79          // Gets first line
80          String inputString = inbuffer.readLine();
81          while (inputString != null)
82          {
83              // Breaks the line into pieces using Tokenizer
84              tokens = new StringTokenizer(inputString, "#");
85
86              // Fields are name#address#yardType#length#width#totalCost
87              String name = tokens.nextToken();
88              String address = tokens.nextToken();
89              String yardType = tokens.nextToken();
90              double length = Double.parseDouble(tokens.nextToken());
91              double width = Double.parseDouble(tokens.nextToken());
92              double totalCost = Double.parseDouble(tokens.nextToken());
93
94              // Creates Customer object and adds it to the ArrayList
95              Customer cust = new Customer(0, name, address, yardType, length, width, totalCost);
96              customers.add(cust);
97
98              // Reads next line
99              inputString = inbuffer.readLine();
100         }
101
102         // Closes the pipe to the file once the records have been read
103         inbuffer.close();
104
105         // Returns the ArrayList
106         return customers;
```

Database Integration

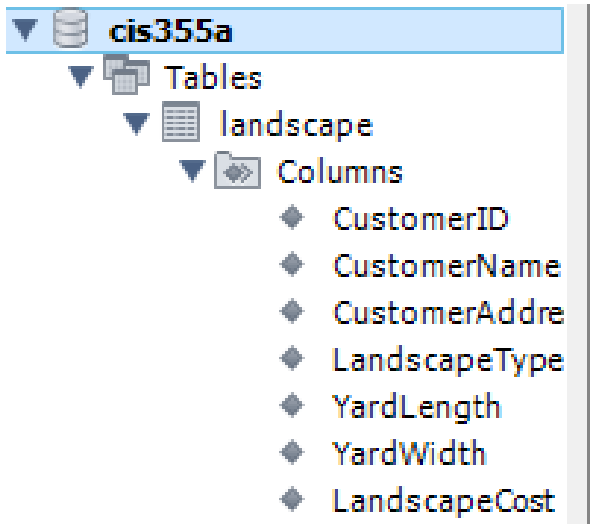
Creation of SQL Database

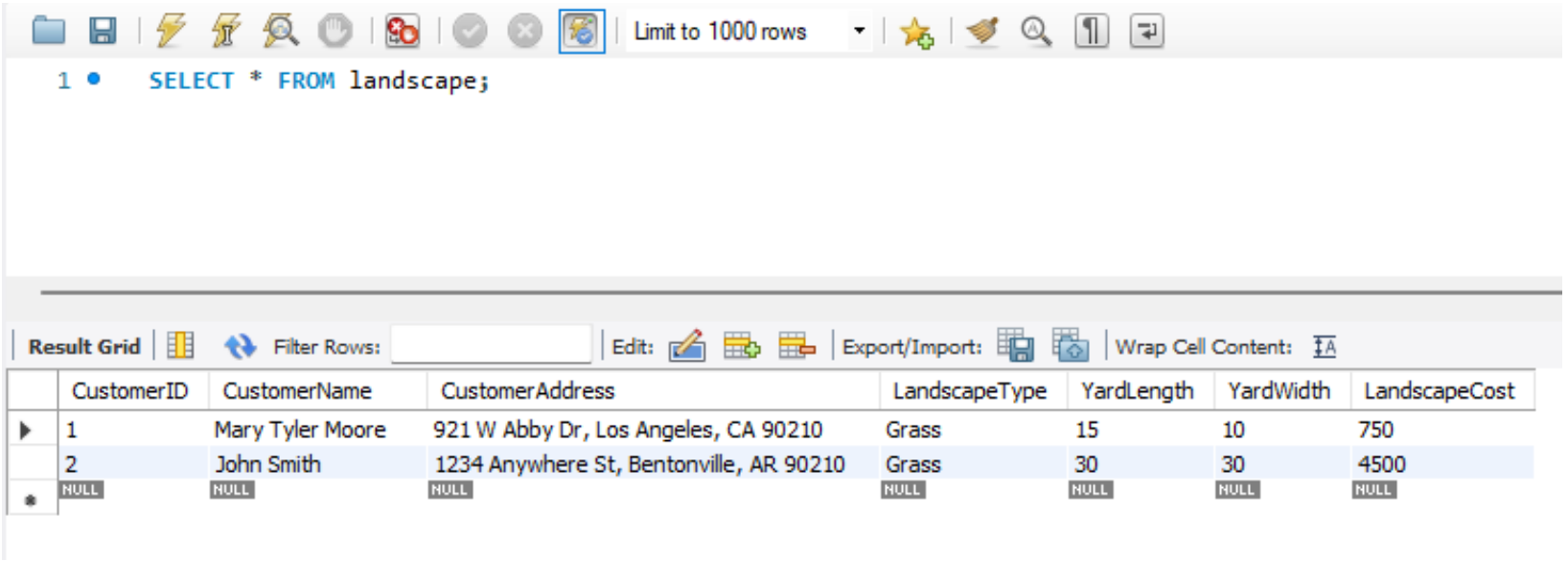
The last portion of this application build involved the creation of a SQL database and integration into the GUI application.

The DataIO class was updated to read and write to a database instead of a file.

Data viewed and retrieved from the database includes the customer name, address, landscape type, yard width, yard length, landscape cost, and an autoincremented CustomerID.

SQL Database





CustomerID	CustomerName	CustomerAddress	LandscapeType	YardLength	YardWidth	LandscapeCost
1	Mary Tyler Moore	921 W Abby Dr, Los Angeles, CA 90210	Grass	15	10	750
2	John Smith	1234 Anywhere St, Bentonville, AR 90210	Grass	30	30	4500
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Updated GUI Functionality

- The “Information” form’s “Submit Order” button and “Submit Order” menu item were both updated to write information to a database.
- The “Customer List” form’s “Load List” button now reads data from a database and displays information in the “Customer List” form’s JList and text area.
- The “Customer List” form’s “Delete Customer” button was updated to remove a client’s information from a database with SQL commands within the application utilizing Prepared Statements to prevent SQL injections.

Database Integration Code

```java
import java.sql.*;
import java.util.ArrayList;


public class DataIO
{
    // Constants
    private final String DATABASE_NAME = "cis355a";
    private final String CONNECTION_STRING =
            "jdbc:mysql://localhost:3306/" + DATABASE_NAME;
    private final String USER_NAME = "root";
    private final String PASSWORD = "devry123";

    // Behaviors
    public void add( Customer cust ) throws ClassNotFoundException, SQLException
    {
        // Check for the driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Connect to the database
        Connection conn = DriverManager.getConnection(CONNECTION_STRING,
                USER_NAME, PASSWORD);

        // Add record
        String strSQL = "INSERT INTO landscape (CustomerName, CustomerAddress, "
                + "LandscapeType, YardLength, YardWidth, LandscapeCost) "
                + "VALUES(?, ?, ?, ?, ?, ?)";
        PreparedStatement pstmt = conn.prepareStatement(strSQL);
        pstmt.setString(1, cust.getName());
        pstmt.setString(2, cust.getAddress());
        pstmt.setString(3, cust.getYardType());
        pstmt.setDouble(4, cust.getLength());
        pstmt.setDouble(5, cust.getWidth());
        pstmt.setDouble(6, cust.getTotalCost());

        // Execute the prepared statement
        pstmt.execute();

        // Close connection
        conn.close();
```

Application Completion

Final Review

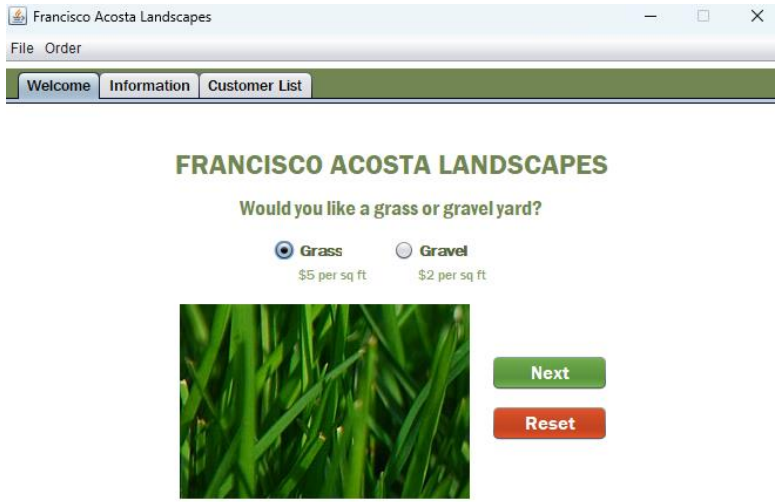
Final assessment of components that were reviewed to create a polished application.

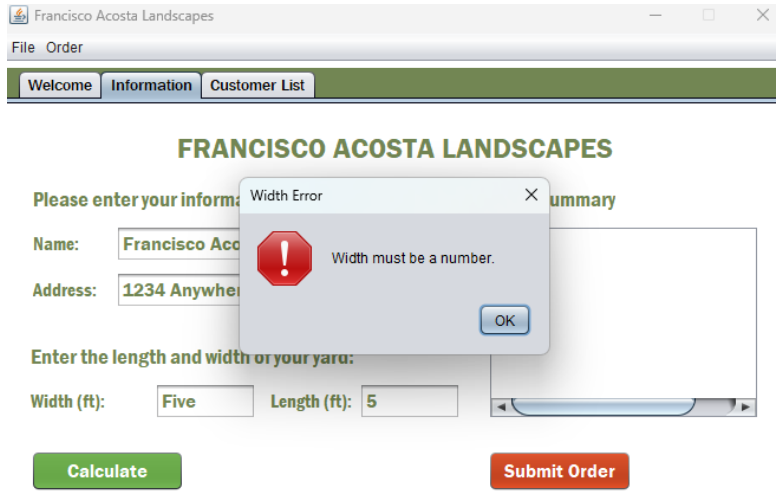
- Classes
- Methods
- Comments with the intention of aiding future revisions
- User experiences such as tabbing through fields
- Variable names
- Indentions
- Exception handling
- Verbiage utilized in the GUI
- Design aesthetic

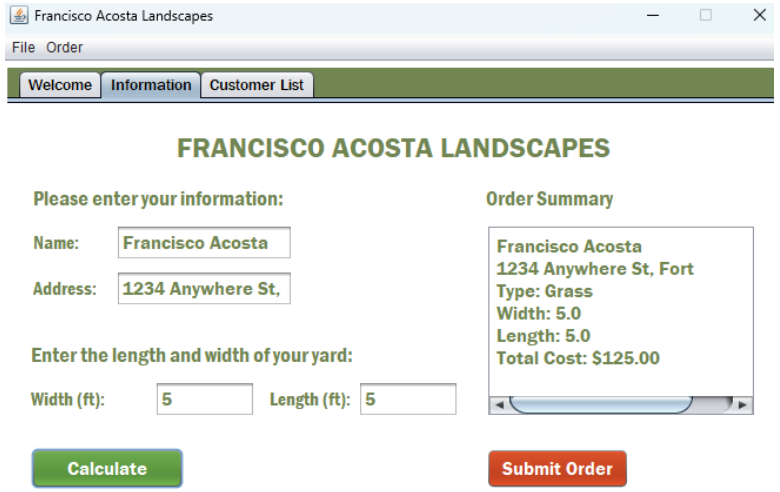
Minor updates:

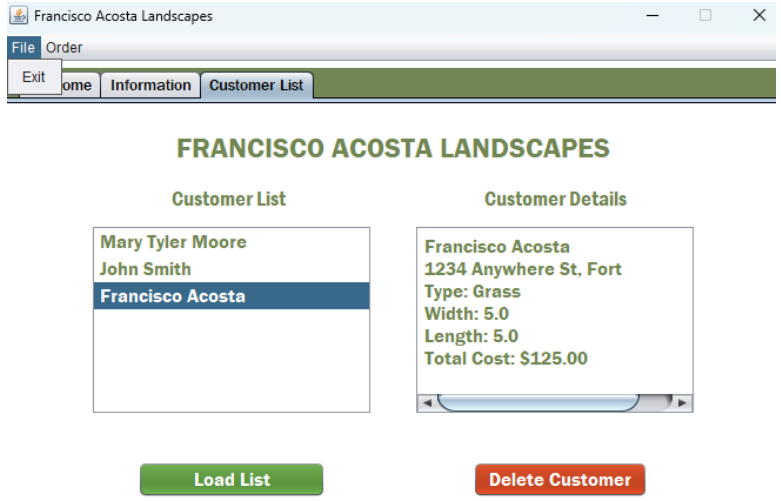
- Change of the “Information” form’s background color to match that of the rest of the application.
- Auto-loading the database list upon the startup of the application.

Completed GUI Application with Database Integration









Challenges

Challenges

The initial challenge was to create an efficiently structured application with well thought-out planning before the development process began.

The next obstacle was understanding the concepts of Java's read and write procedures, as well as its integration into to a SQL database.

Debugging is an expected impediment when developing any application, but persistence and creative problem solving were key to my success.

The last and biggest challenge was creating a safe cohesive application that creates a good user experience with a pleasing design aesthetic that does not hinder efficiency.

Career Skills

Career Skills Gained

- Java syntax and IDE experience
- Creating GUIS’s and debugging within Java
- Creating Wireframe and UML diagrams
- Utilizing the three-tier software development model
- Reading and writing to databases within an application
- Application development from conception to completion
- Exception handling
- Creating a good user experience while not sacrificing efficiency

Conclusion

Conclusion

This Landscaping Java GUI application build served its purpose of keeping track of the necessary information via a SQL database.

Stages of planning, designing and development were followed to create a smooth user experience.

The final product produced a visually pleasing and efficient application while still meeting the desired requirements. This was accomplished by incorporating the three-tier architecture of presentation, business rules and data storage.